# VBC

## *VIBE Control*

Complete User Manual & Command Reference

*Version control for vibe coders.*

*Auto-flags every save. Commit when ready. Zero ceremony.*

Created by **Pratham Kumar Uikey**

github.com/pratham1kruk

# 1. What is VBC?

**VBC (VIBE Control)** is a lightweight, local version control tool built for vibe coders — developers who work in fast, iterative, flow-state sessions. It watches your files and automatically records every save as a **flag**. When something works, you commit it. When you hit a milestone, you drop a checkpoint.

Unlike Git, VBC requires **zero ceremony**. No `git add`. No staging area. No forced commit messages. Every save is automatically classified and recorded. You stay in the flow.

## 1.1 The Two Layers

| Layer | Who sets it | What it does |
|---|---|---|
| **March Flags** | Watcher (auto) | Classify every file save — MF+, mf+, CF, MF- |
| **Commit Flags** | You (manual) | Mark milestones — WF (file), UF (project), CKP (snapshot) |

## 1.2 VBC vs Git

| Aspect | Git | VBC |
|---|---|---|
| Intent | Team collaboration + code shipping | Track your own session history |
| Save trigger | Manual: git add + git commit | Automatic: every file save |
| Unit of work | Commit with message (required) | Flag (auto-classified) |
| Rollback | git checkout, git revert | vbc reset we/us flag, vbc reset ckp |
| Stable point | git tag | vbc plot ckp |
| Export | git archive | vbc clone ckp1 as v1.zip |
| Learning curve | Steep | Zero — just save files |

**TIP** Use VBC during your coding session. Push to Git when you are ready to share.

## 1.3 How Data is Stored

Everything lives in `.vbc/` at your project root. No database, no binary blobs — plain text you can open in any editor.

**Project structure**

```
your-project/
├── .vbc/
│   ├── header.json      ← version counter + init time
│   ├── snapshot.prt     ← all flag entries (WF/UF/MF+/mf+/CF/MF-)
│   └── stver.prt        ← checkpoint store (full file snapshots)
└── .vbcignore           ← patterns VBC will NOT track
```

# 2. Flag Visual Language

Every change in VBC is recorded as a typed, colored flag. Once you know the symbols, you can read your entire history at a glance.

| Symbol | Flag | Meaning |
|---|---|---|
| ● | `MF<n>+` | March File Forward — full file replaced. Triggered on first save or when >75% of lines change. |
| ○ | `mf<n>+` | Man March Forward — partial edit. Some lines changed but less than 75%. |
| ◆ | `CF<n>` | Code File Forward — copy event. File content matches a previously tracked snapshot. |
| ◀ | `MF<n>-` | March File Backward — revert. File shortened or reverted toward an earlier state. |
| ✓ | `WF<n>` | We Flag — single file commit. Collapses all march flags for that file into one entry. |
| ✓ | `UF<n>` | Us Flag — project-wide commit. Collapses march flags across every tracked file. |
| ⬟ | `ckp<n>` | Checkpoint — full project snapshot. All files saved to stver.prt as a restore point. |

## 2.1 Auto-Classification Rules

When the watcher detects a file save, it applies these rules in order:

| Rule | Classified as |
|---|---|
| First ever save for this file | MF+ |
| More than 75% of lines changed | MF+ |
| Some lines changed (less than 75%) | mf+ |
| Content identical to last snapshot | CF |
| File shorter / reverted to older state | MF- |

## 2.2 Flag Numbering

Each flag type is numbered independently per file. `MF3+` means the third full-replace event on that file. `WF2` means the second time you committed it. Numbers never reset unless you reinitialize the repo.

## 2.3 Flag Lifecycle (WE Flag Collapse)

When you run `vbc plot we flag <file>`, all march flags for that file are **purged** and replaced with a single `WF` entry. The terminal shows exactly which flags were collapsed.

**Flag collapse example**

```
# Before WE flag:
  src/auth.py  -->  MF1+  mf2+  CF3  MF4+  mf5+

# After: vbc plot we flag src/auth.py -ms "JWT done"
  src/auth.py  -->  WF1

  Terminal: "Collapsed: MF1+ mf2+ CF3 MF4+ mf5+"
```

# 3. Installation

## 3.1 Requirements

- **Node.js** v16 or later (for source install)
- Windows 10/11, Linux, macOS, or WSL
- VS Code 1.85+ (for the extension — optional)

## 3.2 Windows — Binary Installer (.exe)

Run `VBC_Installer.exe` — a wizard with license, directory selection, and optional VS Code extension install. No Node.js required. The binary has the runtime bundled.

> **NOTE** The binary installer ships vbc.exe — your source code is compiled in and not exposed.

## 3.3 Linux / macOS / WSL — Source

**Terminal**

```
bash install.sh
```

Installs to `~/.vbc-cli` and symlinks `vbc` to `/usr/local/bin`.

## 3.4 Linux — Binary Installer

**Terminal**

```
# Place both files in the same folder:
#   vbc-linux-x64-v1.0.0
#   vbc-install.sh

bash vbc-install.sh             # install
bash vbc-install.sh --update    # update
bash vbc-install.sh --uninstall # remove
```

## 3.5 VS Code Extension

**Install extension**

```
.\install-extension.bat   # Windows
bash install-extension.sh  # Linux / WSL
```

Then reload VS Code: `Ctrl+Shift+P` → **Developer: Reload Window**

## 3.6 Verify Installation

**Any terminal**

```
vbc help
# or
vbc --version
```

## 3.7 Update to New Version

**Terminal**

```
vbc update C:\Downloads\vbc_unified   # Windows
vbc update /home/user/vbc_unified      # Linux / WSL


# Then re-install the extension:
.\install-extension.bat
```

# 4. Quick Start

From zero to your first checkpoint in under 2 minutes:

**Quick start session**

```
# 1. Go to your project folder
cd my-project

# 2. Initialize VBC
vbc init

# 3. Start the watcher in background
vbc watch --bg

# 4. Code normally and save files
#    Watcher auto-assigns:
#    MF1+  src/main.py   (full replace)
#    mf2+  src/utils.py  (partial edit)

# 5. When a file is working — commit it
vbc plot we flag src/main.py -ms "login done"

# 6. When everything is working — commit all
vbc plot us flag -ms "session 1 complete"

# 7. Drop a stable checkpoint
vbc plot ckp -ms "v1.0 -- auth + dashboard"

# 8. Check your history
vbc log
vbc status
```

# 5. Command Reference

## 5.1 vbc init

Initialize a VBC repository in the current directory.

**`vbc init`**
*Creates .vbc/ folder and .vbcignore in the current directory.*

**What gets created:**

- `.vbc/header.json` — version counter, init timestamp, project root
- `.vbc/snapshot.prt` — rolling flag log (starts empty)
- `.vbc/stver.prt` — checkpoint store (starts empty)
- `.vbcignore` — default ignore patterns in project root

**Example:**

```
cd my-project
vbc init


# Output:
# ✓  Initialized empty vbc repository
# .vbc/   .vbcignore created
```

## 5.2 vbc watch

Start the file watcher. Monitors all project files and auto-assigns march flags on every save.

**`vbc watch`**
*Start watcher in foreground. Press Ctrl+C to stop.*

**`vbc watch --bg`**
*Start as a background daemon. Runs even after terminal is closed.*

**`vbc watch --stop`**
*Stop the background daemon.*

**`vbc watch --status`**
*Check whether the daemon is running and show its PID.*

**`vbc watch --tail`**
*Live-tail the background daemon log file (.vbc/watch.log).*

### Auto-classification (applied per save):

| Condition | Flag assigned |
|---|---|
| First ever save for this file | MF+ (yellow ●) |
| More than 75% of lines changed | MF+ (yellow ●) |
| Some lines changed | mf+ (amber ○) |
| Content matches a previous snapshot | CF (purple ◆) |
| File reverted / shortened | MF- (red ◂) |

### Examples:

```
# See flags appear in real time
vbc watch

  ● MF1+   src/main.py          10:23:41   42 lines replaced
  ○ mf2+   src/utils.py         10:24:02   +3 -1
  ◆ CF1    src/config.py        10:24:15   copy event
  ◂ MF1-   src/main.py          10:25:01   reverted

# Run silently in background
vbc watch --bg

# Check if it is running
vbc watch --status
  # Watcher running  PID 12345

# Follow the log
vbc watch --tail

# Stop it
vbc watch --stop
```

**TIP** Use --bg at the start of every session so you never miss a flag even if you close the terminal.

## 5.3 vbc flag

Manually set a march flag without the watcher running. Useful when `vbc watch` is not active.

**`vbc flag mf+ <file>`**

*Full file snapshot — MF+ flag*

`-ms "message"` Attach a message to this flag

**`vbc flag mf <file>`**

*Partial edit snapshot — mf+ flag*

**`vbc flag cf <file>`**

*Copy-point snapshot — CF flag*

**`vbc flag mf- <file>`**

*Interactive revert — opens arrow-key menu of all snapshots for this file*

**Examples:**

```
# Manual full-replace flag
vbc flag mf+ src/utils.py
vbc flag mf+ src/utils.py -ms "added retry logic"

# Interactive revert — choose which snapshot to restore
vbc flag mf- src/main.py
  # Arrow keys to select:
  #   > MF3+  10:25  (current)
  #     MF2+  10:18
  #     MF1+  09:54
```

## 5.4 vbc plot

Set commit-level flags or a checkpoint. These are always set manually — the watcher never sets them.

**`vbc plot we flag <file> [-ms "message"]`**

*Commit a single file -> WF flag. Collapses all march flags for that file.*

`<file>` Relative path to the file to commit

`-ms "message"` Optional message attached to this WF flag

**`vbc plot us flag [-ms "message"]`**

*Commit all tracked files -> UF flag on each. Collapses march flags project-wide.*

`-ms "message"` Optional message for all UF flags

**`vbc plot ckp [-ms "message"]`**

*Full project checkpoint -> CKP. Snapshots every file into stver.prt.*

`-ms "message"` Describe this checkpoint (highly recommended)

**What happens on WE flag:**

- All MF+, mf+, CF, MF- march flags for that file are purged
- A single WF entry is written with timestamp + optional message

- Terminal prints a collapse summary

**What happens on CKP:**

- Full content of every tracked file is saved to stver.prt
- ckp count in header.json is incremented
- A lightweight entry is written to snapshot.prt for visibility in vbc log

**Examples:**

```
# Commit a single file
vbc plot we flag src/auth.py
vbc plot we flag src/auth.py -ms "JWT login complete"

# Commit all files at once
vbc plot us flag
vbc plot us flag -ms "all API routes done"

# Save a checkpoint
vbc plot ckp
vbc plot ckp -ms "v1.0 -- login + dashboard stable"

# Typical end-of-session workflow
vbc plot us flag -ms "session 3 done"
vbc plot ckp    -ms "v2.0 stable"
```

## 5.5 vbc reset

Restore files to a previous committed state. Reset never deletes flags — only file content is restored.

**`vbc reset we flag <file>`**

*Restore one file to its latest WF state.*

**`vbc reset we flag <file> -at WF<n>`**

*Restore one file to a specific WF number.*

`-at WF2` Target WF number to restore to

**`vbc reset us flag`**

*Restore all tracked files to their latest UF state.*

**`vbc reset us flag -at UF<n>`**

*Restore all files to a specific UF number.*

**`vbc reset ckp -at ckp<n>`**

*Restore the ENTIRE project to a checkpoint. -at is required.*

`-at ckp1` Checkpoint label to restore to (check vbc log for available ones)

**Examples:**

```
# Oops -- undo changes to one file since last commit
vbc reset we flag src/api.py

# Go back to a specific commit on one file
vbc reset we flag src/api.py -at WF2

# Restore entire project to last UF
vbc reset us flag

# Restore entire project to specific UF
vbc reset us flag -at UF3

# Restore project to checkpoint ckp1
vbc reset ckp -at ckp1

# After reset -- re-commit once you are satisfied
vbc plot we flag src/api.py -ms "restored to WF2"
```

**WARNING** vbc reset ckp -at ckp1 overwrites your current files. Run vbc plot ckp first if you want to preserve your current state.

## 5.6 vbc clone

Export a checkpoint as a zip archive. The zip contains the exact file state at that checkpoint.

**`vbc clone ckp<n> as <filename>.zip`**

*Export checkpoint n as a zip file.*

**Examples:**

```
vbc clone ckp1 as v1.zip
vbc clone ckp3 as project-march-backup.zip
vbc clone ckp2 as stable-before-refactor.zip

# Then restore it with:
vbc reconstruct v1.zip as project-v1
```

## 5.7 vbc reconstruct

Restore a project from a cloned zip archive into a new directory.

**`vbc reconstruct <file>.zip as <dir>`**

*Unpack a vbc zip (or any standard zip) into a new directory.*

**Examples:**

```
vbc reconstruct v1.zip as project-v1
vbc reconstruct stable-before-refactor.zip as restored
vbc reconstruct project-march-backup.zip as backup
```

---

## 5.8 vbc construct

Export your codebase as a readable document. Two modes: complete (c3) or selective (scc).

**`vbc construct <dir> as c3.<txt|pdf|docx>`**

*C3 - Complete Codebase Copy. Folder structure + every file's full code.*

**`vbc construct <dir> with [f1,f2,...] as scc.<txt|pdf|docx>`**

*SCC - Selective Codebase Copy. Only the listed files are included in the body.*

| | |
|---|---|
| `<dir>` | Source directory (use . for current directory) |
| `with [f1,f2]` | Comma-separated list of relative file paths |
| `c3 / scc` | Mode: complete or selective |
| `.txt/.pdf/.docx` | Output format |

**Document structure:**

- **Header** — project name, date, file count, generated by VBC
- **Folder Structure** — full ASCII tree of the project
- **File Contents** — each file path (bold) followed by its full code

**Examples:**

```
# Full codebase copy as plain text
vbc construct . as c3.txt

# Full codebase as Word document (bold paths, monospace code)
vbc construct . as c3.docx

# Full codebase as PDF
vbc construct myapp as c3.pdf

# Only specific files (selective)
vbc construct . with [src/auth.py,src/db.py] as scc.txt
vbc construct . with [app/main.js,app/routes.js,app/utils.js] as scc.docx

# Output is saved as:
#   myapp_c3_2025-03-26.txt
#   myapp_scc_2025-03-26.docx
```

## 5.9 vbc log

Show the flag history — all files or filtered to one file. Newest flags shown first.

**`vbc log`**
*Show full flag history across all tracked files.*

**`vbc log <file>`**
*Show flag history for a specific file only.*

**Examples:**

```
vbc log
vbc log src/main.py
vbc log app/routes/auth.js

# Sample output:
#   ✓ WF2    src/auth.py     14:32   "JWT complete"
#   ● MF3+   src/auth.py     14:28
#   ○ mf2+   src/auth.py     14:21
#   ● MF1+   src/auth.py     14:10
```

## 5.10 vbc status

Show repository stats at a glance.

**`vbc status`**
*Shows version, file count, flag counts, latest checkpoint, watcher state.*

```
vbc status

# Output:
#   Project  : /home/user/my-project
#   Version  : 147
#   Init     : 2025-03-10
#   Files    : 12 tracked
#   WF       : 24   UF: 6   CKP: 3
#   Latest   : ckp3  "v2.0 stable"
#   Watcher  : running (PID 4821)
```

## 5.11 vbc update

Update the CLI from a newly extracted folder in-place. No reinstall needed.

**`vbc update <path>`**
*Sync CLI from new source folder. On WSL, updates both WSL and Windows sides.*

**Examples:**

```
# Windows
vbc update C:\Downloads\vbc_unified

# Linux / WSL
vbc update /home/user/vbc_unified

# WSL also accepts Windows paths
vbc update C:\Downloads\vbc_unified

# After updating CLI, also update the extension:
.\install-extension.bat
```

## 5.12 vbc help

Show help for all commands or get detailed help for a specific command.

**`vbc help`**
*Show overview of all commands.*

**`vbc help <command>`**
*Show detailed help for one command.*

```
vbc help
vbc help watch
vbc help plot
vbc help reset
vbc help construct
```

# 6. .vbcignore

VBC creates a `.vbcignore` file in your project root on `vbc init`. The watcher reads it on startup. Any file or folder matching a pattern is completely invisible to VBC.

## 6.1 Syntax

- One pattern per line
- Lines starting with # are comments
- `dir/` — trailing slash matches everything inside that directory
- `*.ext` — wildcard matches by extension or pattern
- `filename` — exact filename match anywhere in the tree

## 6.2 Default Patterns

```
# .vbcignore

# Dependency folders
node_modules/
.git/
__pycache__/
.venv/
venv/

# Build output
dist/
build/
out/
.next/

# Lock files
package-lock.json
yarn.lock
pnpm-lock.yaml

# Generated files
*.log
*.pyc
*.min.js
*.min.css
*.map
```

**WARNING** The watcher only reads .vbcignore at startup. If you edit it, restart with: vbc watch --stop then vbc watch

# 7. VS Code Extension

## 7.1 Features

- **Auto-start watcher** — starts vbc watch when you open a VBC repo folder
- **VBC Graph** — live SVG tree of your project history, left to right
- **Diff viewer** — click any flag node to see before/after content
- **Status bar** — shows latest flag and watcher state at all times
- **Keyboard shortcuts** — plot flags without leaving the keyboard
- **Command Palette** — all VBC commands via Ctrl+Shift+P

## 7.2 Keyboard Shortcuts

| Shortcut | Action |
|---|---|
| Ctrl+Shift+W (Cmd+Shift+W) | Plot WE Flag on active file |
| Ctrl+Shift+U (Cmd+Shift+U) | Plot US Flag on all files |
| Ctrl+Shift+K (Cmd+Shift+K) | Plot Checkpoint |
| Ctrl+Shift+G (Cmd+Shift+G) | Open VBC Graph (full tab) |

## 7.3 VBC Graph

Open from the activity bar sidebar or `Ctrl+Shift+G`. The graph is a live SVG tree growing **left to right**:

**Graph layout**

```
ROOT  ----  src/  ----  main.py  ----  MF1+  --  mf2+  --  WF1
            |           utils.py  ---  WF1
            |
            components/  ----  App.jsx  ----  mf1+  --  WF1

   [ckp1]  "v1 stable"     <-- gold diamond on root spine
   [ckp2]  "v2 dashboard"
```

**Node shapes:**

| Shape | Color | Meaning |
|---|---|---|
| Large filled circle | Blue | ROOT node |
| Diamond | Gold | CKP checkpoint on root spine |
| Rounded square | Slate | Folder internal node |
| Hollow circle | Light blue | File leaf node |
| Filled small circle | Green/Blue | WF or UF commit flag |
| Hollow small circle | Yellow/Red/Purple | March flags (MF+/mf+/CF/MF-) |

**Interactions:**

- **Click any flag node** — diff panel opens with before/after content
- **Click a CKP diamond** — popup: Reset project / Clone as zip / Make c3 doc
- **Hover any node** — tooltip with type, timestamp, stats, message

**Search and filter:**

- **Search box** — type filename or flag label. Matching nodes highlight, others fade. Canvas scrolls to first match.
- **Enter key** — jump to next match. Press again to cycle through all hits.
- **Filter buttons** — WF / UF / MF+ / mf+ / CF / MF- / CKP. Click to show only those types. Multiple can be active.
- **Clear button** — reset all filters and search.

## 7.4 Settings

| Setting | Default | Description |
|---|---|---|
| vbc.autoStartWatcher | true | Auto-start watcher when VBC repo is opened |
| vbc.statusBarEnabled | true | Show latest flag in status bar |
| vbc.vbcCliPath | "" | Custom path to vbc.js (blank = auto-detect) |

## 7.5 Command Palette

Press `Ctrl+Shift+P` and type **VBC**:

| Command | Action |
|---|---|
| VBC: Plot WE Flag | Commit active file |
| VBC: Plot US Flag | Commit all files |
| VBC: Plot Checkpoint | Save checkpoint |
| VBC: Reset WE Flag | Restore active file to last WF |
| VBC: Reset US Flag | Restore all files to last UF |
| VBC: Open Graph | Open VBC Graph in full tab |
| VBC: Show Flag Log | Show flag history in Output panel |
| VBC: Show Status | Show repo stats in Output panel |
| VBC: Start Watcher | Start the file watcher |
| VBC: Stop Watcher | Stop the file watcher |
| VBC: Init Repository | Initialize VBC in current workspace |

# 8. Workflow Examples

## 8.1 Typical Vibe Session

**Typical session**

```
# Start of session
cd my-project
vbc watch --bg

# ... code for 2 hours ...
# Watcher auto-records:
#   MF1+ MF2+ mf3+ mf4+ CF5  across various files

# Login feature done
vbc plot we flag src/auth.py    -ms "JWT login done"

# Dashboard done
vbc plot we flag src/dashboard.py -ms "chart rendering fixed"

# Session over -- commit all and checkpoint
vbc plot us flag -ms "session 3 complete"
vbc plot ckp     -ms "v1.0 -- auth + dashboard"

# Stop watcher
vbc watch --stop
```

## 8.2 Something Broke -- Restore a File

**Restore workflow**

```
# api.py is broken after edits
vbc reset we flag src/api.py
# Restored to last WF on that file

# Need to go back further
vbc reset we flag src/api.py -at WF2

# Restore entire project to a checkpoint
vbc reset ckp -at ckp1
```

## 8.3 Export a Stable Version

**Export workflow**

```
# See your checkpoints
vbc log

# Export ckp2 as a zip
vbc clone ckp2 as project-v2.zip

# Give to someone else or restore later
```

```
vbc reconstruct project-v2.zip as project-v2-restored
```

## 8.4 Export Codebase as Document

**Codebase export**

```
# Full codebase for review / archiving
vbc construct . as c3.docx

# Only the core logic files
vbc construct . with [src/auth.py,src/db.py,src/routes.py] as scc.pdf
```

# 9. Tips and Best Practices

- **Always start with vbc watch --bg** — background mode means you never miss a flag even if you close the terminal.
- **WF early and often** — whenever something small works, commit it. WF is cheap and reversible.
- **CKP at real milestones** — "v1 login", "API complete". These are hard restore points stored forever.
- **Always add -ms on CKP** — you will thank yourself when reading vbc log in two weeks.
- **Keep .vbcignore tight** — node_modules, dist, .venv should always be ignored. A noisy tree is hard to read.
- **Use the graph** — open Ctrl+Shift+G to see your project history as a tree. Each branch length shows how much work a file has had.
- **VBC is not a Git replacement** — use VBC during sessions to capture every step. Push to Git when ready to share.

# 10. Quick Reference Cheatsheet

| Command | Description |
|---|---|
| vbc init | Initialize repository in current folder |
| vbc watch | Start watcher (foreground) |
| vbc watch --bg | Start watcher (background daemon) |
| vbc watch --stop | Stop background watcher |
| vbc watch --status | Check if watcher is running |
| vbc watch --tail | Follow background log |
| vbc flag mf+ <file> [-ms "msg"] | Set march flag manually |
| vbc flag mf- <file> | Interactive revert (arrow-key menu) |
| vbc plot we flag <file> [-ms "msg"] | Commit single file -> WF |
| vbc plot us flag [-ms "msg"] | Commit all files -> UF |
| vbc plot ckp [-ms "msg"] | Full project snapshot -> CKP |
| vbc reset we flag <file> | Restore file to latest WF |
| vbc reset we flag <file> -at WF2 | Restore file to specific WF |
| vbc reset us flag | Restore all files to latest UF |
| vbc reset us flag -at UF3 | Restore all files to specific UF |
| vbc reset ckp -at ckp1 | Restore entire project to checkpoint |
| vbc clone ckp<n> as <n>.zip | Export checkpoint as zip |
| vbc reconstruct <n>.zip as <dir> | Restore from zip |
| vbc construct . as c3.txt | Export full codebase as text |
| vbc construct . as c3.docx | Export full codebase as Word doc |
| vbc construct . with [f1,f2] as scc.pdf | Export selected files as PDF |
| vbc log | Full flag history |
| vbc log <file> | Flag history for one file |
| vbc status | Repo stats |
| vbc update <path> | Update CLI from new folder |

| Command | Description |
|---|---|
| vbc help / vbc help <cmd> | Help |

*VBC - VIBE Control*

**Created by Pratham Kumar Uikey**

github.com/pratham1kruk